DAX Expressions

Function Type	DAX Output Type	Create as...
Aggregation (SUM, AVG)	One result per report	**Measure**
Logical (IF, AND)	One value per row	**Calculated Column**
Text (LEFT, CONCATENATE)	One value per row	**Calculated Column**
Statistical (STDEV.P)	One result per report	**Measure**
Math (ABS, ROUND)	Depends on context	Usually a **Column**
Filter (CALCULATE, FILTER)	Summary values with context	**Measure**
Time Intelligence	Aggregate by date	**Measure**

Aggregation Functions

- **SUM()**
  Adds up all values in a column.
  - Syntax: `SUM(<column>)`
  - Example: `Total Sales = SUM(Sales[SalesAmount])`
- **AVERAGE()**
  Calculates the average value in a column.
  - Syntax: `AVERAGE(<column>)`
  - Example: `Average Sales = AVERAGE(Sales[SalesAmount])`
- **COUNT()**
  Counts the number of non-null values in a column.
  - Syntax: `COUNT(<column>)`
  - Example: `Sales Count = COUNT(Sales[SalesAmount])`
- **COUNTROWS()**
  Counts the number of rows in a table.
  - Syntax: `COUNTROWS(<table>)`
  - Example: `Row Count = COUNTROWS(Sales)`
- **MIN()**
  Returns the smallest value in a column.
  - Syntax: `MIN(<column>)`
  - Example: `Min Sales = MIN(Sales[SalesAmount])`
- **MAX()**
  Returns the largest value in a column.
  - Syntax: `MAX(<column>)`
  - Example: `Max Sales = MAX(Sales[SalesAmount])`

- **DISTINCTCOUNT()**
  Counts the number of distinct values in a column.
    - Syntax: `DISTINCTCOUNT(<column>)`
    - Example: `Unique Customers = DISTINCTCOUNT(Sales[CustomerID])`

Date and Time Functions

- **TODAY()**
  Returns the current date.
    - Syntax: `TODAY()`
    - Example: `Current Date = TODAY()`
- **YEAR()**
  Extracts the year from a date.
    - Syntax: `YEAR(<date>)`
    - Example: `Year = YEAR(Sales[Date])`
- **MONTH()**
  Extracts the month from a date.
    - Syntax: `MONTH(<date>)`
    - Example: `Month = MONTH(Sales[Date])`
- **DATE()**
  Creates a date from individual year, month, and day values.
    - Syntax: `DATE(<year>, <month>, <day>)`
    - Example: `Custom Date = DATE(2025, 5, 4)`
- **DATEDIFF()**
  Calculates the difference between two dates.
    - Syntax: `DATEDIFF(<start_date>, <end_date>, <unit>)`
    - Example: `Days Between = DATEDIFF(Sales[StartDate], Sales[EndDate], DAY)`

Logical Functions

- **IF()**
  Checks a condition and returns one value if true, another if false.
  - Syntax: `IF(<logical_test>, <value_if_true>, <value_if_false>)`
  - Example: `Sales Category = IF(Sales[SalesAmount] > 1000, "High", "Low")`
- **AND()**
  Checks if two or more conditions are true.
  - Syntax: `AND(<condition1>, <condition2>)`
  - Example: `High Sales and Discount = AND(Sales[SalesAmount] > 1000, Sales[Discount] > 0.2)`
- **OR()**
  Checks if at least one condition is true.
  - Syntax: `OR(<condition1>, <condition2>)`
  - Example: `High Sales or Discount = OR(Sales[SalesAmount] > 1000, Sales[Discount] > 0.2)`
- **NOT()**
  Reverses the result of a logical test.
  - Syntax: `NOT(<logical_test>)`
  - Example: `Not High Sales = NOT(Sales[SalesAmount] > 1000)`
- **SWITCH()**
  Evaluates an expression and returns a result based on different conditions.
  - Syntax: `SWITCH(<expression>, <value1>, <result1>, <value2>, <result2>, ..., <else_result>)`
  - Example:
    ```
    Sales Category = SWITCH(TRUE(), Sales[SalesAmount] > 1000, "High", Sales[SalesAmount] > 500, "Medium", "Low")
    ```

Text Functions

- **CONCATENATE()**
  Joins two text strings into one.
  - Syntax: `CONCATENATE(<text1>, <text2>)`
  - Example: `Full Name = CONCATENATE(Employee[FirstName], Employee[LastName]`
- **LEFT()**
  Extracts a specified number of characters from the left side of a text string.
  - Syntax: `LEFT(<text>, <num_characters>)`
  - Example: `First 3 Characters = LEFT(Employee[Name], 3)`
- **RIGHT()**
  Extracts a specified number of characters from the right side of a text string.
  - Syntax: `RIGHT(<text>, <num_characters>)`
  - Example: `Last 3 Characters = RIGHT(Employee[Name], 3)`
- **LEN()**
  Returns the length of a text string.
  - Syntax: `LEN(<text>)`
  - Example: `Name Length = LEN(Employee[Name])`
- **UPPER()**
  Converts a text string to uppercase.
  - Syntax: `UPPER(<text>)`
  - Example: `Uppercase Name = UPPER(Employee[Name])`
- **LOWER()**
  Converts a text string to lowercase.
  - Syntax: `LOWER(<text>)`
  - Example: `Lowercase Name = LOWER(Employee[Name])`

Filter Functions

- **CALCULATE()**
  Evaluates an expression in a context modified by filters.
  - Syntax: `CALCULATE(<expression>, <filter1>, <filter2>, ...)`
  - Example:
    ```
    Sales 2023 = CALCULATE(SUM(Sales[SalesAmount]), Sales[Year] = 2023)
    ```
- **FILTER()**
  Returns a table that represents a subset of another table.
  - Syntax: `FILTER(<table>, <expression>)`
  - Example:
    ```
    Filtered Sales = FILTER(Sales, Sales[SalesAmount] > 1000)
    ```
    ```
    CALCULATE(SUM(SALES_DAX[SALESAMOUNT]),FILTER(Sales_DAX, Sales_DAX[SalesAmount] > 1000))
    ```
- **ALL()**
  Removes filters from a column or table.
  - Syntax: `ALL(<column|table>)`
  - Example:
    ```
    Total Sales All = CALCULATE(SUM(Sales[SalesAmount]), ALL(Sales[Year]))
    ```
- **ALLEXCEPT()**
  Removes all filters except for the specified columns.
  - Syntax: `ALLEXCEPT(<table>, <column1>, <column2>, ...)`
  - Example:
    ```
    Sales Except Year = CALCULATE(SUM(Sales[SalesAmount]), ALLEXCEPT(Sales, Sales[Product]))
    ```

Time Intelligence Functions

- **TOTALYTD()**
  Calculates the year-to-date total for an expression.
  - Syntax: `TOTALYTD(<expression>, <dates>, <year_end_date>)`
  - Example: `Total Sales YTD = TOTALYTD(SUM(Sales[SalesAmount]), Sales[Date])`
- **PREVIOUSMONTH()**
  Returns a table with values from the previous month.
  - Syntax: `PREVIOUSMONTH(<dates>)`
  - Example:
    ```
    Sales Last Month =
    CALCULATE(SUM(Sales[SalesAmount]),
    PREVIOUSMONTH(Sales[Date]))
    ```
- **SAMEPERIODLASTYEAR()**
  Returns a table with values from the same period in the previous year.
  - Syntax: `SAMEPERIODLASTYEAR(<dates>)`
  - Example:
    ```
    Sales Same Period Last Year =
    CALCULATE(SUM(Sales[SalesAmount]),
    SAMEPERIODLASTYEAR(Sales[Date]))
    ```

Statistical Functions

- **STDEV.P()**
  Calculates the standard deviation of a population.
  - Syntax: `STDEV.P(<column>)`
  - Example: `Sales StdDev = STDEV.P(Sales[SalesAmount])`
- **VAR.P()**
  Calculates the variance of a population.
  - Syntax: `VAR.P(<column>)`
  - Example: `Sales Variance = VAR.P(Sales[SalesAmount])`

Mathematical Functions

- **ABS()**
  Returns the absolute value of a number.
  - Syntax: `ABS(<number>)`
  - Example: `Absolute Value = ABS(Sales[Profit])`
- **ROUND()**
  Rounds a number to a specified number of digits.
  - Syntax: `ROUND(<number>, <num_digits>)`
  - Example: `Rounded Sales = ROUND(Sales[SalesAmount], 2)`
- **POWER()**
  Returns the result of a number raised to a specified power.
  - Syntax: `POWER(<number>, <power>)`
  - Example: `Sales Squared = POWER(Sales[SalesAmount], 2)`

Math Functions

- **CEILING()**
  Rounds a number up, away from zero, to the nearest multiple of a given significance.
  - Syntax: `CEILING(<number>, <significance>)`
  - Example: `Ceiling Sales = CEILING(Sales[SalesAmount], 10)`
- **FLOOR()**
  Rounds a number down, towards zero, to the nearest multiple of a given significance.
  - Syntax: `FLOOR(<number>, <significance>)`
  - Example: `Floor Sales = FLOOR(Sales[SalesAmount], 10)`
- **MOD()**
  Returns the remainder when one number is divided by another.
  - Syntax: `MOD(<number>, <divisor>)`
  - Example: `Sales Mod = MOD(Sales[SalesAmount], 10)`

- **RAND()**
  Returns a random decimal number between 0 and 1.
    - Syntax: `RAND()`
    - Example: `Random Value = RAND()`
- **RANDBETWEEN()**
  Returns a random integer between a specified minimum and maximum value.
    - Syntax: `RANDBETWEEN(<min>, <max>)`
    - Example: `Random Sales = RANDBETWEEN(100, 1000)`

Date Dimension table

```
DateTable =
ADDCOLUMNS(
        CALENDAR(DATE(2015,1,1), DATE(2030,12,31)),
        "Year", YEAR([Date]),
        "Month Number", MONTH([Date]),
        "Month Name", FORMAT([Date], "MMMM"),
        "Quarter", "Q" & FORMAT([Date], "Q"),
        "Weekday", FORMAT([Date], "dddd"),
        "Year-Month", FORMAT([Date], "YYYY-MM")
)
```